

#13 Query Methods, Sorting & Pagination

Query Methods, Sorting & Pagination | Derived queries, custom queries, sorting and pagination

These concepts are used when we want to retrieve data from the database in different ways instead of fetching all records.

Why Do We Need Query Methods?

Suppose we have a Student table.

id	name	email	age	city
1	Ummed	ummed@gmail.com	25	Jaipur
2	Rahul	rahul@gmail.com	23	Delhi
3	Priya	priya@gmail.com	24	Jaipur
4	Amit	amit@gmail.com	25	Mumbai

Sometimes we need:

- Find student by email
- Find students from Jaipur
- Find students whose age is greater than 23
- Sort students by name
- Show only 10 records per page

For these operations Spring Data JPA provides:

1. Derived Query Methods
 2. Custom Queries
 3. Sorting
 4. Pagination
-

Project Structure

Controller
|
Service
|
Repository
|
Database

Step 1: Create Entity

```
@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private Integer age;
    private String city;

    // Getters and Setters
}
```

Generated Table

```
CREATE TABLE students(
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255),
    email VARCHAR(255),
    age INT,
    city VARCHAR(255)
);
```

Derived Query Methods

Derived Query

Spring automatically generates SQL query from method name.

No SQL writing required.

Example 1: Find By Email

Repository

```
public interface StudentRepository
    extends JpaRepository<Student, Long> {

    Student findByEmail(String email);

}
```

Usage

```
Student student = repository.findByEmail("ummed@gmail.com");
```

Generated SQL

```
SELECT * FROM students WHERE email='ummed@gmail.com';
```

Example 2: Find By Name

```
List<Student> findByName(String name);
```

Generated SQL

```
SELECT * FROM students WHERE name='Ummed';
```

Example 3: Find By City

```
List<Student> findByCity(String city);
```

Generated SQL

```
SELECT * FROM students WHERE city='Jaipur';
```

Example 4: Multiple Conditions

```
List<Student> findByNameAndCity( String name, String city);
```

SQL

```
SELECT * FROM students WHERE name='Ummed' AND city='Jaipur';
```

Example 5: OR Condition

```
List<Student> findByNameOrCity( String name, String city);
```

SQL

```
SELECT * FROM students WHERE name='Ummed' OR city='Jaipur';
```

Example 6: Greater Than

```
List<Student> findByAgeGreaterThan( Integer age);
```

Usage

```
repository.findByAgeGreaterThan(22);
```

SQL

```
SELECT * FROM students WHERE age > 22;
```

Example 7: Less Than

```
List<Student> findByAgeLessThan( Integer age);
```

SQL

```
SELECT * FROM students WHERE age < 22;
```

Example 8: Between

```
List<Student> findByAgeBetween( Integer start, Integer end);
```

SQL

```
SELECT * FROM students WHERE age BETWEEN 20 AND 25;
```

Example 9: Like

```
List<Student> findByNameContaining( String keyword);
```

Usage

```
repository.findByNameContaining("mm");
```

SQL

```
SELECT * FROM students WHERE name LIKE '%mm%';
```

Example 10: Starts With

```
List<Student> findByNameStartingWith( String prefix);
```

SQL

```
SELECT * FROM students WHERE name LIKE 'Um%';
```

Example 11: Ends With

```
List<Student> findByNameEndingWith( String suffix);
```

SQL

```
SELECT * FROM students WHERE name LIKE '%ed';
```

Example 12: Order By

```
List<Student> findByCityOrderByAgeDesc( String city);
```

SQL

```
SELECT * FROM students WHERE city='Jaipur' ORDER BY age DESC;
```

Common Derived Query Keywords

Keyword	Example
And	findByNameAndCity
Or	findByNameOrCity
GreaterThan	findByAgeGreaterThan
LessThan	findByAgeLessThan
Between	findByAgeBetween
Like	findByNameLike
Containing	findByNameContaining
StartingWith	findByNameStartingWith
EndingWith	findByNameEndingWith
IsNull	findByEmailIsNull
OrderBy	findByCityOrderByAgeDesc

Custom Queries

Sometimes method names become very long.

Example:

```
findByNameAndCityAndAgeGreaterThanAndEmailContaining(...)
```

Not readable.

For complex queries use **@Query**.

JPQL Query

```
@Query("SELECT s FROM Student s WHERE s.city = :city")  
List<Student> getStudentsByCity( @Param("city") String city);
```

Usage

```
repository.getStudentsByCity("Jaipur");
```

JPQL Explanation

```
SELECT s FROM Student s
```

Means:

```
SELECT * FROM students;
```

Note: JPQL uses

Student (Entity Name) Not students (Table Name)

Multiple Conditions

```
@Query("""SELECT s FROM Student s WHERE s.city=:city AND  
s.age>:age""")  
List<Student> getStudents( String city, Integer age);
```

Generated SQL

```
SELECT * FROM students WHERE city='Jaipur' SemanticContext.AND  
age>22;
```

Native Query

Sometimes we want actual SQL.

```
@Query(value = "SELECT * FROM students WHERE city=?1",  
nativeQuery = true)  
List<Student> getByCity(String city);
```

SQL

```
SELECT * FROM students WHERE city='Jaipur';
```

When To Use JPQL?

Use when: **Entity based querying**

```
SELECT s FROM Student s
```

When To Use Native SQL?

Use when:

- Complex SQL
- Database-specific functions

```
SELECT * FROM students;
```

Sorting

Suppose data:

id	name
3	Rahul
1	Ummed
2	Amit

Need:

Amit
Rahul
Ummed

Use Sorting.

Repository

No extra code required.

```
public interface StudentRepository extends  
JpaRepository<Student, Long> {  
}
```

Sort Ascending

```
List<Student> students = repository.findAll(Sort.by("name"));
```

SQL

```
SELECT * FROM students ORDER BY name ASC;
```

Sort Descending

```
List<Student> students = repository.findAll(  
Sort.by("name").descending());
```

SQL

```
SELECT * FROM students ORDER BY name DESC;
```

Multiple Column Sorting

```
Sort sort = Sort.by("city").and(Sort.by("age").descending());  
repository.findAll(sort);
```

SQL

```
SELECT * FROM students ORDER BY city ASC, age DESC;
```

Pagination

Problem

Suppose table has: **1,000,00 Students**

If we do: **repository.findAll();**

All records loaded. Bad Performance

Solution

Pagination: Fetch only required records.

Example:

Page 1 -> 10 records

Page 2 -> 10 records

Page 3 -> 10 records

Page Request

```
Pageable pageable = PageRequest.of(0, 5);
```

Meaning

- 0 = First Page
- 5 = Records Per Page

Fetch Page Data

```
Page<Student> page = repository.findAll(pageable);
```

SQL

```
SELECT * FROM students LIMIT 5 OFFSET 0;
```

Second Page

```
PageRequest.of(1,5);
```

SQL

```
SELECT * FROM students LIMIT 5 OFFSET 5;
```

Third Page

```
PageRequest.of(2,5);
```

SQL

```
SELECT * FROM students LIMIT 5 OFFSET 10;
```

Understanding Page Object

```
Page<Student> page = repository.findAll(pageable);
```

Useful Methods

- `page.getContent();`
- `page.getTotalPages();`
- `page.getTotalElements();`
- `page.getNumber();`
- `page.getSize();`

Example

```
System.out.println(page.getTotalPages());  
System.out.println(page.getTotalElements());  
System.out.println(page.getContent());
```

Pagination + Sorting Together

```
Pageable pageable = PageRequest.of(0, 5,  
Sort.by("name").ascending());
```

Usage

```
Page<Student> page = repository.findAll(pageable);
```

SQL

```
SELECT * FROM students ORDER BY name ASC LIMIT 5 OFFSET 0;
```

API Example

Repository

```
public interface StudentRepository extends  
JpaRepository<Student, Long> {  
  
}
```

Service

```
public Page<Student> getStudents(int page, int size) {  
  
    Pageable pageable = PageRequest.of(page, size);  
    return repository.findAll(pageable);  
}
```

Controller

```
@GetMapping("/students")  
public Page<Student> getStudents( @RequestParam int page,  
@RequestParam int size) {
```

```
return service.getStudents(page, size);  
}
```

API: GET /students?page=0&size=5

Response

```
{  
  "content": [  
    {  
      "id": 1,  
      "name": "Ummed"  
    }  
  ],  
  "totalPages": 10,  
  "totalElements": 50,  
  "size": 5,  
  "number": 0  
}
```

byteXL

Interview Questions

Q1. What are Derived Query Methods?

Methods where Spring generates SQL automatically based on method name.

findByEmail(String email)

Q2. What is @Query?

Used for custom JPQL or SQL queries.

@Query("SELECT s FROM Student s")

Q3. Difference Between JPQL and Native Query?

JPQL	Native SQL
------	------------

Uses Entity	Uses Table
Database Independent	Database Specific
Portable	Less Portable

Q4. Why Pagination?

To fetch limited records and improve performance.

Q5. Difference Between List and Page?

List	Page
Only Data	Data + Metadata
No Pagination Info	Total Pages, Size, Elements

byteXL